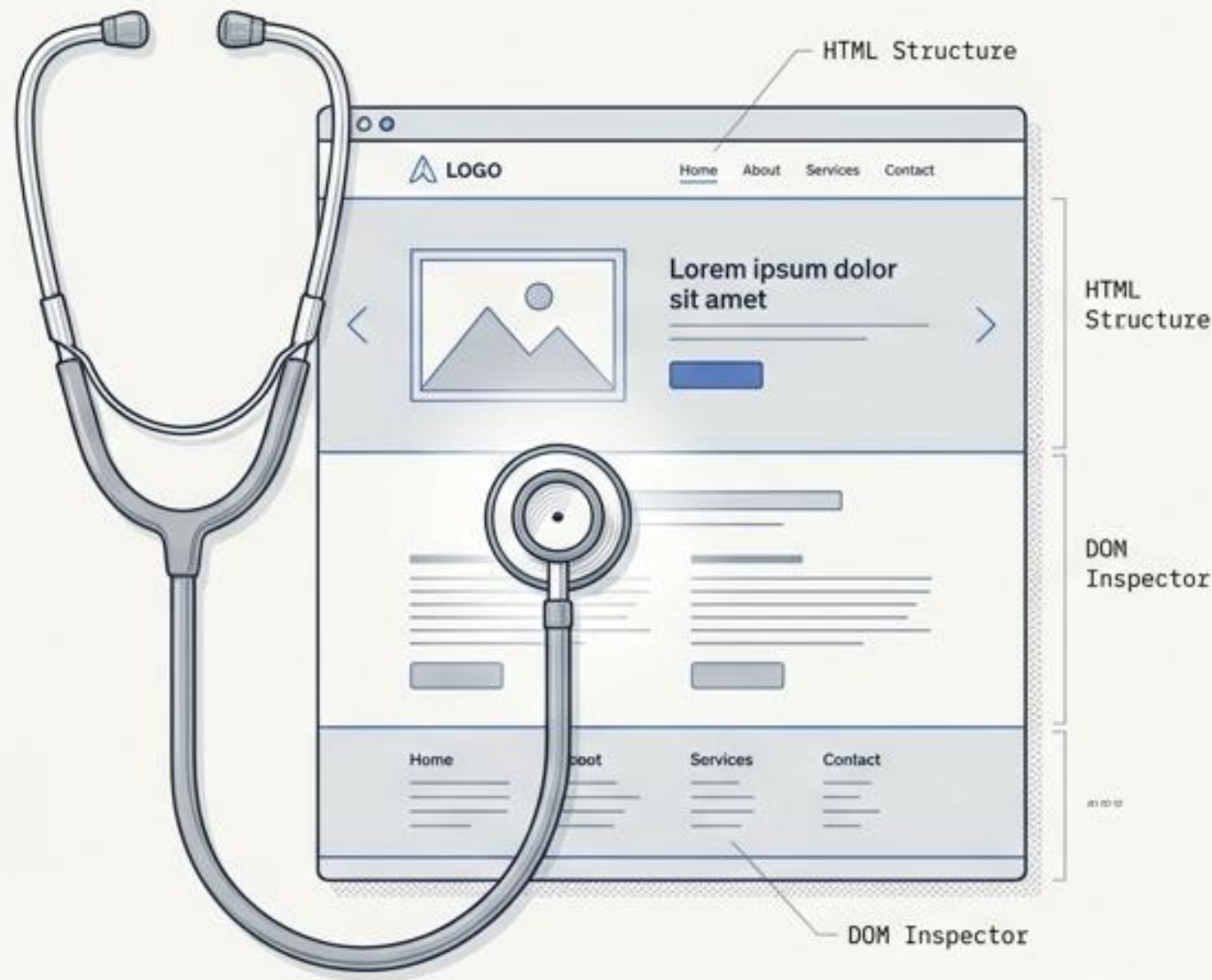


Devs Tools para Testers

Taller Práctico de Herramientas de Desarrollo para QA





Definición: Utilidades integradas en el navegador para inspeccionar y editar código en tiempo real. Son el estetoscopio del tester para ver qué pasa dentro de la aplicación.

¿Qué son las Dev Tools y por qué son útiles para QA?



Precisión

Adiós al 'no funciona'. Hola al `error 500` o `ID` mal configurado en tus reportes de bugs.



Pruebas Caja Gris

Visibilidad de la interacción Frontend/Backend sin acceder al código fuente del servidor.



Ahorro de Tiempo

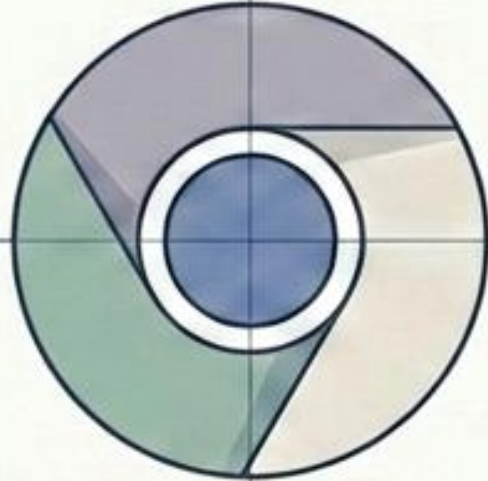
Modifica texto o CSS en vivo para validar cambios visuales al instante, sin esperar a un desarrollador.



Simulación

Prueba cómo se ve la web en resoluciones móviles o con conexiones lentas desde tu PC.

Navegadores Compatibles



Google Chrome

Acceso: F12 o Clic
derecho > Inspeccionar



Microsoft Edge

Acceso: F12 o Clic
derecho > Inspeccionar



Mozilla Firefox

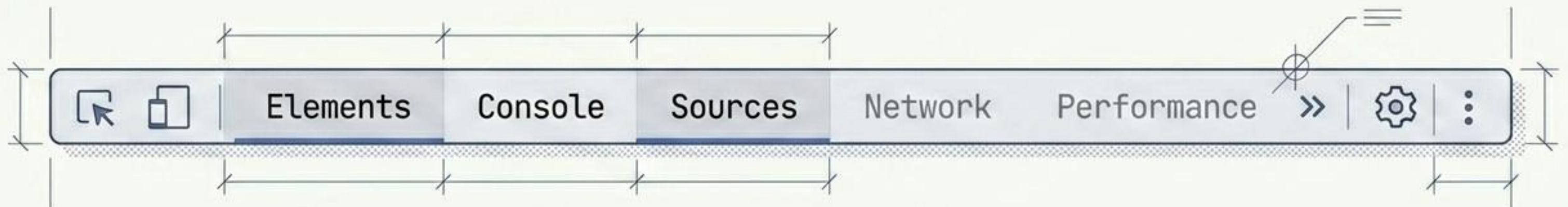
Acceso: F12 o Clic
derecho > Inspeccionar
elemento



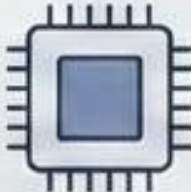
Safari

Acceso: Activa 'Menú
Desarrollo' en
Preferencias > Avanzado

Recorrido por las diferentes solapas



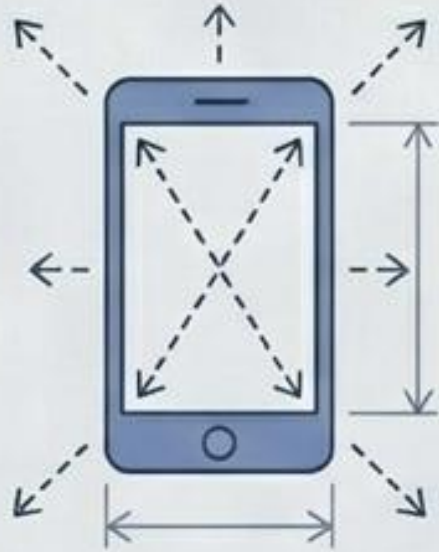
Mobile Tools: Entendiendo los Entornos

 Simulador (DevTools)	 Emulador	 Dispositivo Real
Imita solo el comportamiento del software (la aplicación).	Intenta imitar tanto el software como el hardware.	La prueba definitiva en hardware físico auténtico.



Alerta QA: Las pruebas en DevTools son una simulación. No reemplazan las pruebas en un teléfono real, ya que no consideran el hardware específico (cámara, sensores reales, rendimiento térmico).

Capacidades del Menú Mobile



El Modo Mobile

Adaptación instantánea de pantalla a resoluciones específicas.



Simulación de Rendimiento y Red

Estrangulamiento de conexión (3G lento, modo Offline).



Capturas de Pantalla

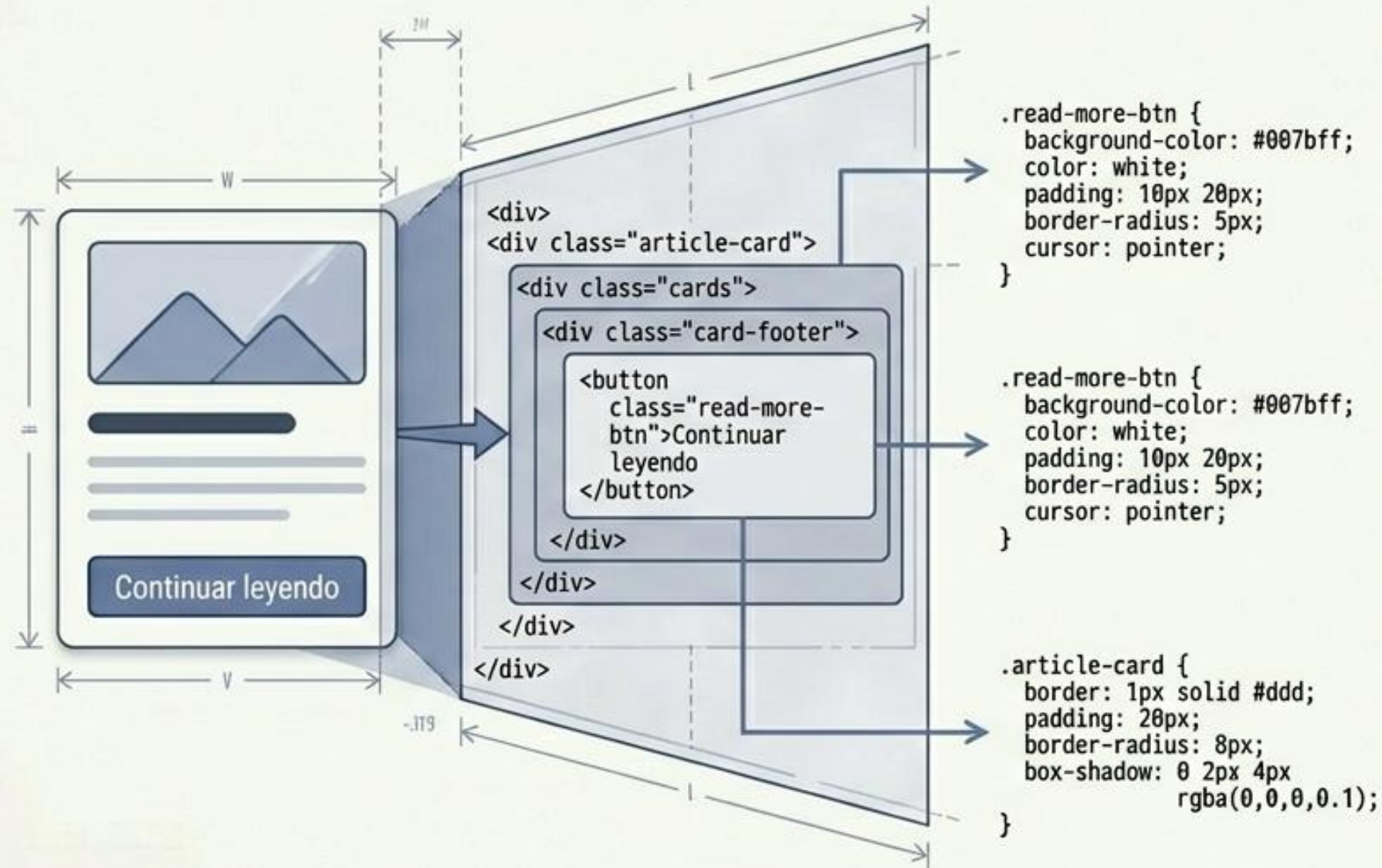
Herramientas nativas precisas para tomar Screenshots.



Hardware Avanzado

Simulación de sensores del dispositivo y Geolocalización virtual.

Solapa de Elements: El Esqueleto Web



Inspección en Tiempo Real
Clic derecho > Inspeccionar.
Se resalta el código HTML en el panel izquierdo y sus reglas CSS en el derecho.

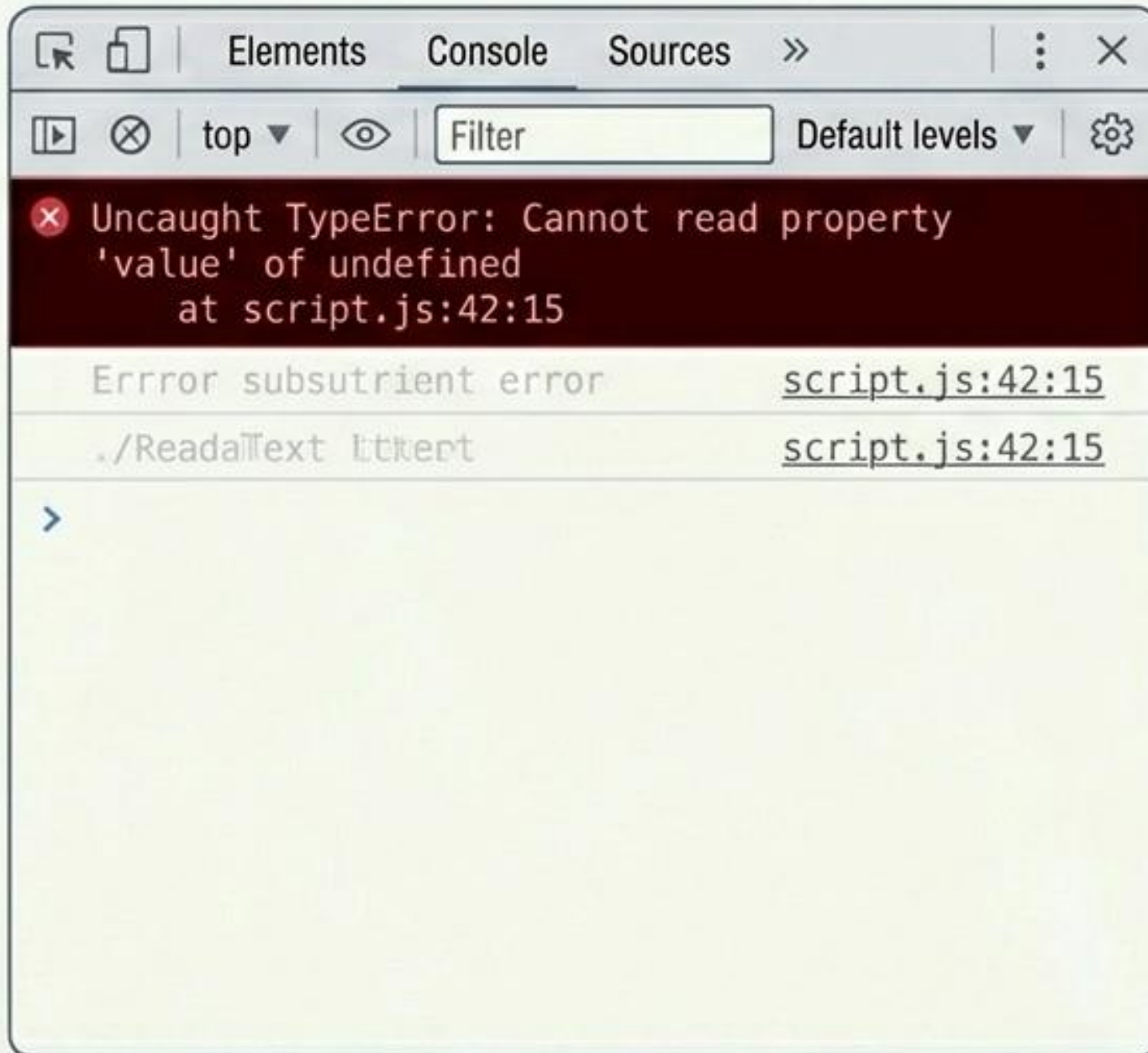


Edición Temporal
Cambia textos (ej. de Continuar leyendo a Seguir leyendo) o estilos visuales al vuelo.



Naturaleza Volátil
Los cambios son solo para depuración local. Al recargar la página, todo vuelve a su estado original.

Solapa de Console



1. Visualización de Logs (Prioridad QA)

Lectura de mensajes de error, advertencias (warnings) e información registrada por el sistema durante el uso.

2. Ejecución de JavaScript

Escribir y correr código directamente en el navegador para interactuar con la página.



Tip de Oro para el Tester:

Si encuentras un error, saca una captura de pantalla del mensaje en rojo en la consola y adjúntala a tu reporte.

Impacto: Ahorra muchísimo tiempo al desarrollador para encontrar la causa raíz.

Actividad Sincrónica: Device Mode

Sistema bajo prueba: <https://cita-ai.vercel.app/login>

1. Simular

Activar dispositivo Sangun S20 Ultra y luego cambiar a Iphone SE

2. Observar

Analizar cómo se transforman, adaptan los botones, textos y formatos.

3. El Reto

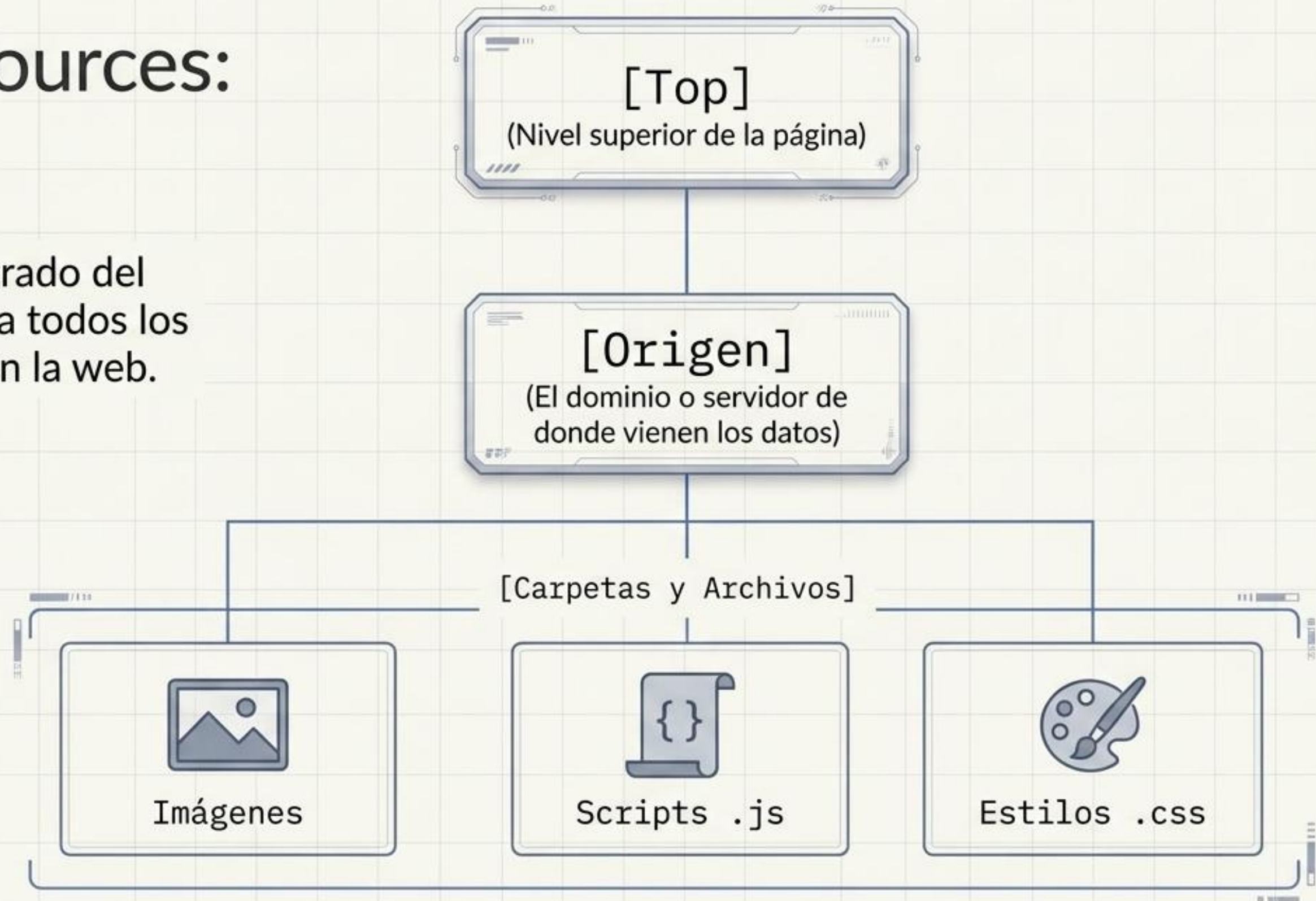
Verificar si algún elemento se pisa con otro, desaparece o queda fuera de lugar.



Objetivo QA: Identificar bugs de diseño responsivo que solo ocurren en resoluciones específicas.

Solapa de Sources: ¿Qué es?

Es el organizador integrado del navegador que muestra todos los archivos que componen la web.



Sources: Utilidad y Debugging para QA

Utilidad Específica



Rastreo de errores:

Un clic en el archivo desde la Consola te lleva a la línea de código exacta del fallo en Sources.



Inspección de recursos:

Ver qué imágenes, archivos JavaScript y hojas de estilo se están cargando realmente.



Verificación CSS:

Abrir archivos para revisar detalles técnicos (ej. `font-family`, colores, márgenes).

Herramientas de Depuración (Debugging)



Breakpoints:

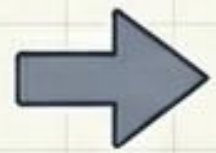
Pausar la ejecución del código en una línea específica para ver el paso a paso.



Watch y Call Stack:

Monitorear variables y entender la secuencia de funciones llamadas.

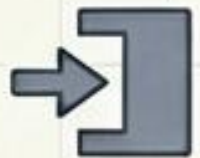
Debuggin - Funcionalidades de la barra de control



Continue: Continúa con la ejecución del código hasta el siguiente breakpoint.



Step over: Ejecuta la siguiente línea de código. Si es una función, la ejecuta y retorna el resultado.



Step into: Ejecuta la siguiente línea de código, pero si es una función “entra” a ejecutarla línea por línea.



Step out: Devuelve el debugger a la línea donde se llamó a la función.



Restart: Reinicia el debugger.



Stop: Frena el debug.

DIFERENCIA ENTRE DEBUGGING Y TESTING

DEBUGGING

1. Proceso deductivo para identificar, localizar y corregir la causa raíz de un error encontrado durante las pruebas
2. Permite dar solución a la falla del código
3. Realizado generalmente por el programador, dev o tester
4. Comienza con la ejecución del código debido a un caso de prueba fallido.

TESTING

1. Proceso donde se comprueba que el sistema funcione de acuerdo a lo esperado. Su objetivo es la búsqueda de errores/defectos
2. Permite identificar fallas del código implementado.
3. Realizado generalmente por el tester
4. Comienza antes de tener el código (Testing estático) o después de que se escribe el código (Testing dinámico)

Gracias por participar del taller
Devs Tools para Tester